Naive Bayes Classifier

- Naive Bayes is a **statistical classification** technique based on **Bayes Theorem**.
- It is one of the **Simplest** Supervised Learning algorithms.
- Naive Bayes classifier is the **fast**, **accurate** and **reliable** algorithm.
- Naive Bayes classifiers have **high accuracy** and **speed on large datasets**.
- Naive Bayes classifier assumes that the effect of a **particular feature** in a class is **independent** of **other features**.
- **For example**, a loan applicant is desirable or **not depending** on his/her **income**, **previous loan** and **transaction history**, **age**, and **location**. Even if these features are interdependent, these features are still considered independently. This assumption simplifies computation, and that's why it is considered as naive.

Math Behind Naïve Bayes

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- **P(h):** the **probability of hypothesis** h being **true** (regardless of the data). This is known as the prior probability of h.
- **P(D):** the **probability of the data** (regardless of the hypothesis). This is known as the prior probability.
- **P(h|D):** the **probability of hypothesis** h given the data D. This is known as **posterior probability**.
- **P(D|h):** the **probability of data** D given that the hypothesis h was **true**. This is known as **posterior probability**.

PLAY-TENNIS EXAMPLE

Weather Temperature Humidity Windy Class				
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

weather	Yes	No	
Sunny	2	3	5/14=0.3571
Overcast	4	0	4/14=0.2857
Rainy	3	2	5/14=0.3571
Total	9/14 = 0.642	5/14=.3571	
temperature	Yes	No	
Hot	2	2	4/14=0.2857
Mild	4	2	6/14=0.4285
Cool	3	1	4/14=0.2857
Total	9/14=0.642	5/14=0.3571	

P(p) = 9/14
P(n) = 5/14

PLAY-TENNIS EXAMPLE 1

Weather Temperature Humidity Windy Class				
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

humidity	Yes	No	
High	3	4	7/14=0.5
Normal	6	1	7/14=0.5
Total	9/14=0.642	5/14=0.3571	
Windy	Yes	No	
True	3	3	6/14=0.4285
False	6	2	8/14=0.571
Total	9/14=0.642	5/14=0.3571	

P(p) = 9/14
P(n) = 5/14

CONTD….

- **Test Data Set:**

Outlook	Temperature	Humidity	Windy	Class
Rain	Hot	High	False	?

- P(X|p)= P(rain|p)*P(hot|p)*P(high|p)*P(false|p)*P(p) = 3/9 * 2/9 * 3/9 * 6/9 * 9/14 = 0.010582
- P(X|n)= P(rain|n)*P(hot|n)*P(high|n)*P(false|n)*P(n) = 2/5 * 2/5 * 4/5 * 2/5 * 5/14 = 0.018286

P(X|n) > P(X|p)

Outlook	Temperature	Humidity	Windy	Class
Rain	Hot	High	False	N (not play)

EXAMPLE

Q: Using Naive Bayes Classification and given table, classify the given tuple (T), also draw the training table for Probabilistic approach.

- Trained Data Set

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

- Test Data Set

Age	Income	Student	Credit_ Rating	Buys_ computer
>40	High	No	Fair	?
<=30	Low	Yes	Fair	?
31…40	Medium	Yes	Excellent	?

Example 2:

RID	Age	Income	Student	Credit Rating	Buys-Computer
1	Youth	High	No	Fair	No
2	Youth	High	No	Excellent	No
3	Middle-aged	High	No	Fair	Yes
4	Senior	Medium	No	Fair	Yes
5	Senior	Low	Yes	Fair	Yes
6	Senior	Low	Yes	Excellent	No
7	Middle-Aged	Low	Yes	Excellent	Yes
8	Youth	Medium	No	Fair	No
9	Youth	Low	Yes	Fair	Yes
10	Senior	Medium	Yes	Fair	Yes
11	Youth	Medium	Yes	Excellent	Yes
12	Middle-aged	Medium	No	Excellent	Yes
13	Middle-aged	High	Yes	Fair	Yes
14	Senior	Medium	No	Excellent	No

New Instance

Age = Youth, Income = Medium, Student = Yes, Credit Rating = Fair

<u>Example 3</u>:

No	Color	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

New Instance

(Color=Green, legs=2, Height=Tall, and Smelly=No)

K-Nearest Neighbors (KNN) Algorithm

K-Nearest Neighbors (KNN) algorithm is a type of **supervised ML** algorithm which can be used for **both classification** as well as **regression** predictive problems. However, it is mainly used for classification predictive problems in industry. The following **two** properties would define KNN well −

- **Lazy Learning algorithm** − KNN is a lazy learning algorithm because it **does not** have a **specialized training** phase and uses all the data for training while classification.

- **Non-Parametric learning algorithm** − KNN is also a non-parametric learning algorithm because it **doesn't assume anything** about the underlying data.

Working of K-Nearest Neighbors Algorithm

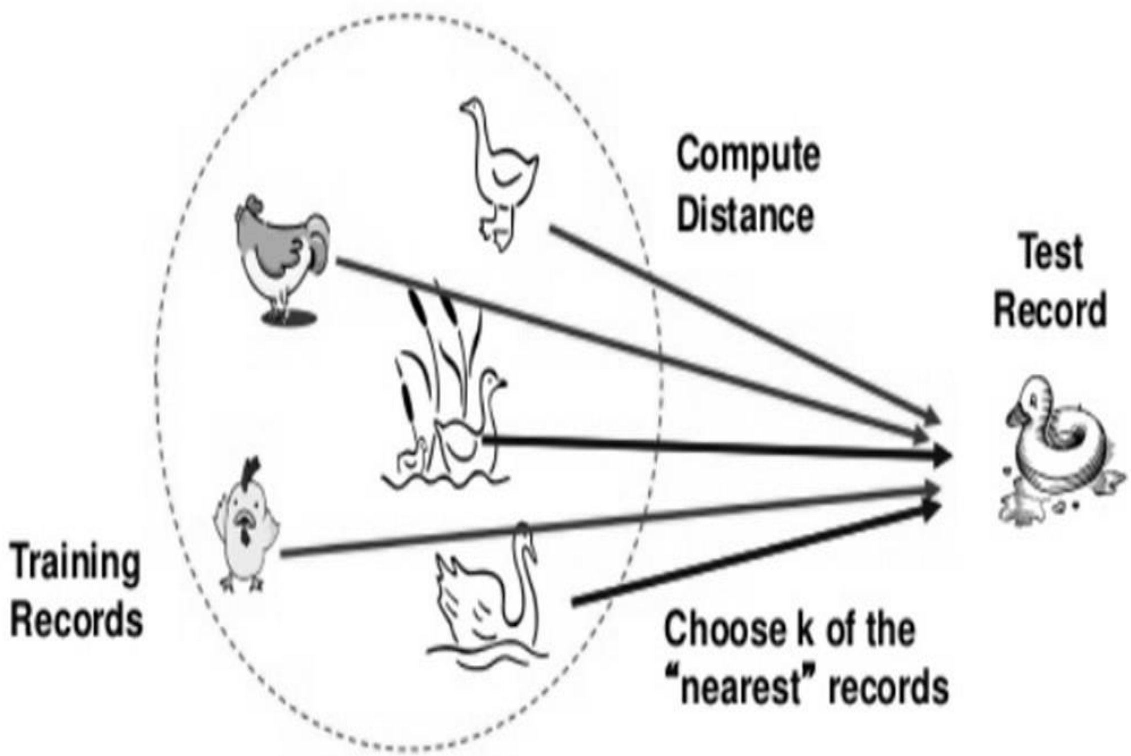
- K-nearest neighbors (KNN) algorithm uses '**feature similarity**' to predict the values of **new data points** which further means that the new data point will be assigned a value based on how closely it matches the points in the **training set**. We can understand its working with the help of following steps −

- **<u>Step 1</u>** − For implementing any algorithm, we need dataset. So, during the first step of KNN, we must **load the training** as well as **test data**.
- **<u>Step 2</u>** − Next, we need to **<u>choose the value of K</u>** i.e. the nearest data points. **<u>K can be any integer</u>**.

Working of K-Nearest Neighbors Algorithm

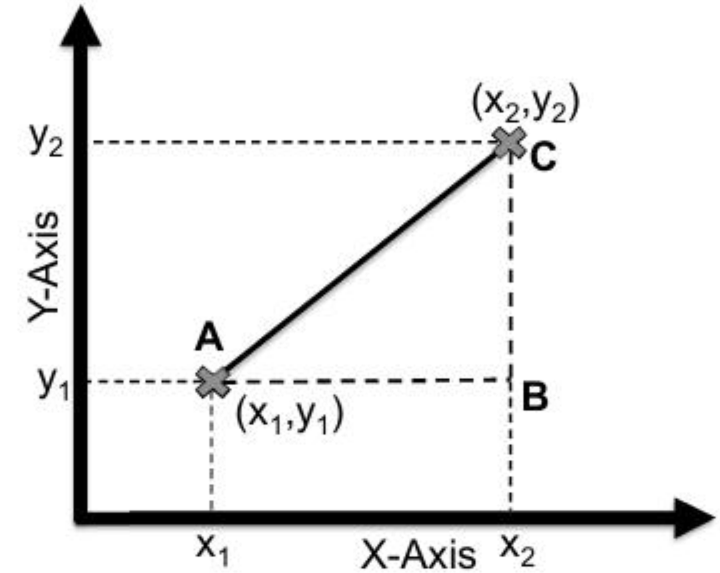
- **Step 3** – For each point in the test data do the following –
  - **Step 3.1** – **Calculate** the **distance** between **test data** and each row of **training data** with the help of any of the method namely: **Euclidean**, **Manhattan** or **Hamming distance**. The most commonly used method to calculate **distance is Euclidean**.
  - **Step 3.2** – Now, based on the distance value, **sort** them in **ascending order**.
  - **Step 3.3** – Next, it will **choose** the **top K rows** from the sorted array.
  - **Step 3.4** – Now, it will **assign** a class to the test point based on **most frequent class** of these rows.
  - **Step 4** – End

Distance Measure:



Eucledian Distance

- Calculate Distance using Pythagoras theorem.



$$AC^2 = AB^2 + BC^2$$

$$AC = \sqrt{AB^2 + BC^2}$$

$$AC = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

Example 1:

We have data from the questionnaires survey (to ask people opinion) and objective testing with **two attributes** (acid **durability** and **strength**) to **classify** whether a special paper tissue is **good** or **not**. Here is **four** training samples

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Y = Classification
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

Now the factory produces a new paper tissue that pass laboratory test with X1 = 3 and X2 = 7. Without another expensive survey, can we guess what the classification of this new tissue is?

- Determine parameter K = number of nearest neighbors (**Suppose K = 3**)
- Calculate the distance between the **query-instance** and all the **training samples**
- Coordinate of query instance is (3, 7), instead of calculating the distance we compute square distance which is faster to calculate (without square root)

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Euclidean Distance to Query Instance (3, 7)
7	7	$d(a_n, b_n) = \sqrt{(3-7)^2 + (7-7)^2} = 4$
7	4	$d(a_n, b_n) = \sqrt{(3-7)^2 + (7-4)^2} = 5$
3	4	$d(a_n, b_n) = \sqrt{(3-3)^2 + (7-4)^2} = 3$
1	4	$d(a_n, b_n) = \sqrt{(3-1)^2 + (7-4)^2} = 3.6$

- **Sort** the **distance** and **determine** nearest neighbors based on the **K-th minimum distance**

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Euclidean Distance to Query Instance (3, 7)	Rank Minimum Distance	Is it included in 3-Nearest neighbors?
7	7	$d(a_n, b_n) = \sqrt{(3-7)^2+(7-7)^2} = 4$	3	Yes
7	4	$d(a_n, b_n) = \sqrt{(3-7)^2+(7-4)^2} = 5$	4	No
3	4	$d(a_n, b_n) = \sqrt{(3-3)^2+(7-4)^2} = 3$	1	Yes
1	4	$d(a_n, b_n) = \sqrt{(3-1)^2+(7-4)^2}$ $= 3.6$	2	Yes

- Gather the category of the nearest neighbors. Notice in the 2nd row last column that the **category of nearest neighbor** (Y) is not included because the **rank of this data** is more than 3.

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Euclidean Distance to Query Instance (3, 7)	Rank Minimum Distance	Is it included in 3-Nearest neighbors?	Y = Category of nearest Neighbor
7	7	$d(a_n, b_n) = \sqrt{(3-7)^2+(7-7)^2} = 4$	3	Yes	Bad
7	4	$d(a_n, b_n) = \sqrt{(3-7)^2+(7-4)^2} = 5$	4	No	-
3	4	$d(a_n, b_n) = \sqrt{(3-3)^2+(7-4)^2} = 3$	1	Yes	Good
1	4	$d(a_n, b_n) = \sqrt{(3-1)^2+(7-4)^2} = 3.6$	2	Yes	Good

- Use simple majority of the category of **nearest neighbors** as the **prediction value** of the query instance
- We have **2 good** and **1 bad**, since 2>1 then we conclude that a new paper tissue that pass laboratory test with X1 = 3 and X2 = 7 is included in **Good category**.

Example 2

- Consider a dataset having two variables: height(cm) & weight(kg) and each point is classified as Normal and Underweight.
- On the basis of the given data we have to classify the below set as Normal or Underweight using KNN (**K=3**)

Weight (X1)	Height (Y1)	Class
57	170	?????

Weight (X1)	Height (Y1)	Class
51	167	Underweight
62	182	Normal
69	176	Normal
64	173	Normal
65	172	Normal
56	174	Underweight
58	169	Normal
57	173	Normal
55	170	Normal

Example 3:

S.No	Name	Age	Income	Loan Decision
1	Ahmed	19	20	**Risky**
2	Sajid	22	18	**Risky**
3	Saleem	25	40	**Safe**
4	Khalid	32	20	**Risky**
5	Akbar	36	19	**Safe**
6	Wahid	55	30	**Safe**
7	Majeed	60	35	**Safe**
8	Sultan	45	45	**Safe**

Mr. Javed Ahmed has applied for loan. His age is 34 and income is 45 K. Being an analyst you are required to asses this case for loan whether it may be Risky or Safe?

Applications of K-NN

- The following are some of the areas in which KNN can be applied successfully –
- **Banking System**
  - KNN can be used in banking system to **predict wether** an individual is fit for loan approval? Does that individual have the characteristics similar to the defaulters one?
- **Calculating Credit Ratings**
  - KNN algorithms can be used to find an individual's credit rating by comparing with the persons having similar traits.

Applications of K-NN

- **Politics**
  - With the help of KNN algorithms, we can **classify a potential voter** into **various classes** like "Will Vote", "Will not Vote", "Will Vote to Party 'PTI', "Will Vote to Party 'PMLN'.
- Other areas in which KNN algorithm can be used are Speech Recognition, Handwriting Detection, Image Recognition and Video Recognition.

Advantages

- It is **very simple algorithm** to understand and interpret.
- It is very **useful for nonlinear data** because there is no assumption about data in this algorithm.
- It is a **versatile algorithm** as we can use it for **classification** as well as **regression**.
- It has **relatively high accuracy**.

Disadvantages

- It is **computationally a bit expensive** algorithm because it stores all the training data.
- **High memory storage** required as compared to other supervised learning algorithms.
- It is very **sensitive to the scale of data** as well as **irrelevant features**.